# Loan Applicant Default Risk Analysis & Prediction

**SQL Server | Python | pandas | scikit-learn | Power BI**
Synthetic-data analysis and a demonstration workflow for prioritising loan-application review.

## Business problem

Historical loan outcomes can help reveal risk patterns, but a review team also needs an actionable view of incoming applications. This project combines descriptive analysis with applicant-level model scores to identify which new applications may warrant closer manual review.

## Scope and definition

| Dataset component | Scope |
|---|---|
| Applications | 30,000 synthetic applications across 12 Indian cities. |
| Historical cohort | 24,000 applications from January 2023 to June 2025: 19,393 approved and 4,607 rejected. |
| New-scoring cohort | 6,000 applications from August 2026, without observed default outcomes. |
| Default label | At least one 90+ days-past-due snapshot within the first 12 months for an issued loan. |

## Method

**1. Prepare in SQL:** validate records, handle duplicates and missing values, join applications with credit history and loan outcomes, and derive application-time features such as obligation and income ratios.

**2. Model in Python:** use pandas for data handling and a scikit-learn pipeline for imputation, scaling, category encoding and classification. Compare Logistic Regression with Random Forest using validation results, then evaluate on a later-period test set.

**3. Score and report:** refit the selected configuration on historical loans with available outcomes, score new applicants and export results to Power BI. Save the fitted pipeline and scoring settings with joblib.

## Intended use

The deliverable is a portfolio demonstration of a review-prioritisation workflow. A priority flag is not a rejection decision, and no realised business savings or reduction in defaults is claimed.

# Historical applications & defaults

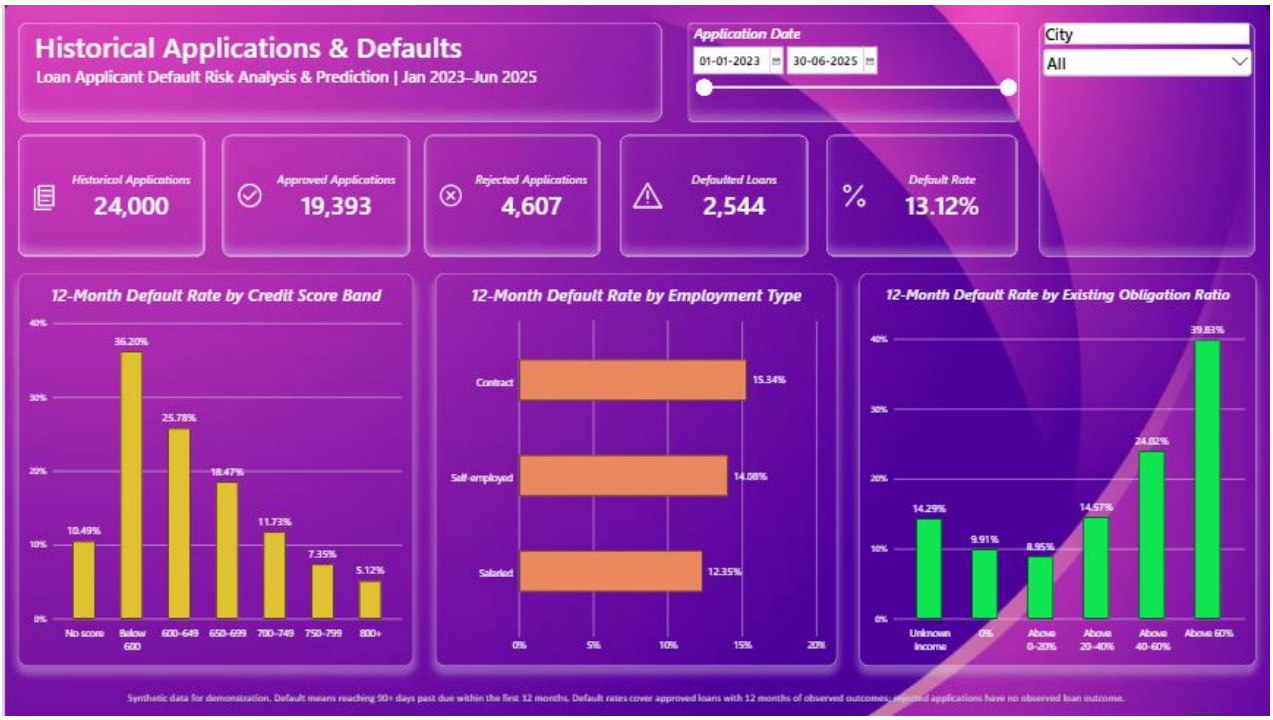



Figure 1. Final historical dashboard, including KPI icons. Screenshot preserved as supplied.

| Historical measure | Result |
|---|---|
| Applications / approved / rejected | 24,000 / 19,393 / 4,607 |
| Defaulted issued loans | 2,544 |
| 12-month default rate | 13.12% of the 19,393 approved loans |

## Patterns visible in the dashboard

**Credit score:** the below-600 band has a 36.20% default rate, compared with 5.12% for scores of 800 or higher. The no-score group is shown separately rather than treated as a zero score.

**Existing obligations:** the above-60% obligation-ratio group has a 39.89% default rate, compared with 8.95% in the above-0%-to-20% group. This suggests affordability deserves attention in review.

**Employment type:** observed default rates are 15.24% for contract applicants, 14.08% for self-employed applicants and 12.35% for salaried applicants.

## Interpretation

These are descriptive associations in synthetic approved-loan data, not causal effects or rules for accepting or rejecting applicants. Segment rates should be considered alongside sample sizes and other application characteristics.

# New applicant review priorities




Figure 2. Final new-applicant dashboard with review status, city workload and applicant detail.

| Scoring output | Result |
|---|---|
| Applicants scored | 6,000 |
| Priority review / standard review | 1,300 / 4,700 |
| Priority review share | 21.67% |
| Mean estimated default probability | 15.38% |
| Priority-review rule | Estimated probability at least 20% |

## How a reviewer would use it

Start with the priority-review list, sort applications by estimated risk and inspect supporting application and credit information. The city chart shows where review volume is concentrated; volume alone does not indicate a higher risk rate.

## Interaction check

A Kolkata filter check returned 504 scored applicants, 115 priority reviews and 389 standard reviews. The priority share was 22.82%, with a mean estimated risk of 15.57%. The visible table and city chart filtered to Kolkata.

## What these numbers do not mean

The 1,300 flagged applications are not confirmed defaults. The 15.38% mean score is not an observed default rate. Probabilities are uncalibrated estimates, and standard-review applicants can still default.

# Model evaluation & conclusions

| Validation model | ROC-AUC | Average precision |
|---|---|---|
| Logistic Regression | 0.7218 | 0.3541 |
| Random Forest | 0.7320 | 0.3685 |

Random Forest was slightly stronger on both validation measures. Its configuration uses 300 trees, a maximum depth of 8 and a minimum of 20 samples per leaf.

| Later-period test metric | Reported result |
|---|---|
| ROC-AUC / average precision | 0.6932 / 0.2683 |
| Precision at the 20% threshold | 29.08% |
| Recall at the 20% threshold | 32.95% |
| F1 score | 0.3089 |

## Evaluation design

The 7,700-loan training pool was split into 6,160 training and 1,540 validation rows using a stratified random split. A separate test contained 3,987 January-June 2025 applications. Another 7,706 historical loans were outside the evaluation timing groups. After evaluation, the final pipeline was refitted on all 19,393 historical loans before scoring August 2026 applications.

## What the test shows

At the chosen threshold, about 29 of every 100 flagged test applicants actually defaulted, while about 33 of every 100 actual defaulters were caught. Later-period performance was weaker than validation performance. ROC-AUC is a ranking measure, not classification accuracy.

## Limitations and next steps

All data are synthetic, and repayment labels exist only for approved, issued loans. Performance on rejected applicants is not established. Calibration, fairness and monetary-loss optimisation were not assessed. Earlier SQL exploration included test-period outcomes, so the full workflow did not retain a completely untouched holdout.

The reported test results belong to the evaluated model, not an independent evaluation of the final refit. Some edited notebook cells have no saved execution output. Next steps are a complete reproducible rerun, a fresh chronological holdout, calibration checks and a review-capacity-based threshold assessment.

## Conclusion

The project demonstrates an integrated SQL-to-Python-to-Power-BI workflow and turns model scores into an understandable review queue. Its strongest contribution is the combined analysis and reporting process; the model remains a limited experimental component.